

KAREN'T CAR RENT

System do obsługi wypożyczalni pojazdów – Karen't

XX

XX

XX

Spis treści

1.	Wymagania użytkownika („historyjka”).....	2
2.	Diagramy przypadków użycia (wymagania funkcjonalne).....	4
3.	Wymagania niefunkcjonalne jako lista	5
4.	Diagram klas – analityczny	6
5.	Diagram klas – projektowy	7
6.	Scenariusz przypadku użycia (jako wypunktowany tekst)	8
7.	Diagram aktywności dla przypadku użycia	10
8.	Diagram stanu dla klasy	11
9.	Projekt GUI	12
9.1	Scenariusz główny.....	12
9.2	Scenariusze alternatywne	15
10.	Omówienie decyzji projektowych i skutków analizy dynamicznej	16

1. Wymagania użytkownika („historyjka”)

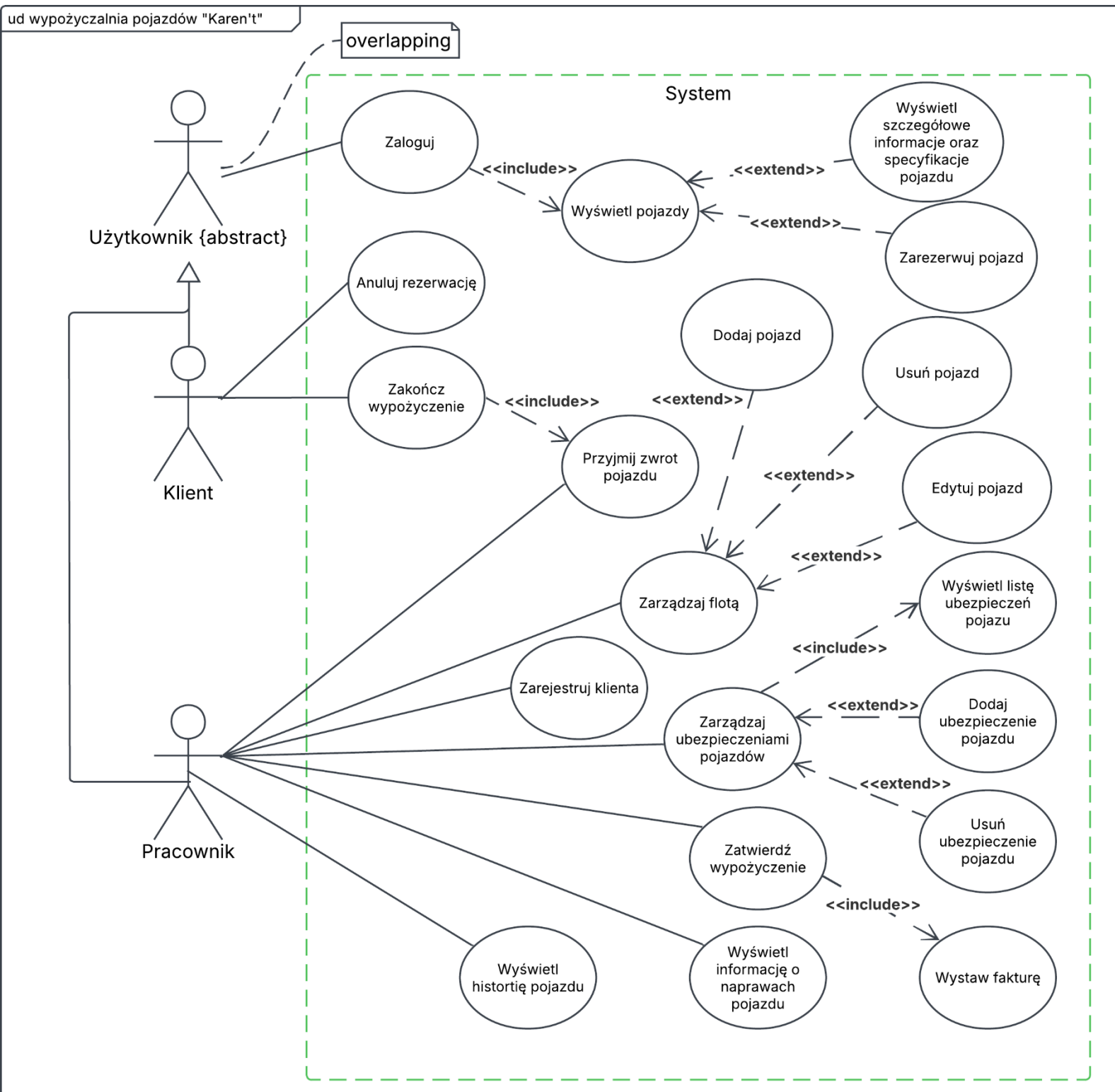
Wypożyczalnia samochodów

System „Karent” jest aplikacją służącą do zarządzania wypożyczalnią pojazdów, która wspiera zarówno pracowników, jak i klientów w realizacji operacji związanych z wynajmem pojazdów, zarządzaniem rezerwacjami, ubezpieczeniami oraz procesami finansowymi.

- 1) Każdy użytkownik systemu posiada konto z unikalnym loginem, hasłem oraz rolą pracownik albo klient.
- 2) Wypożyczalnia zatrudnia wielu pracowników. Dla każdego z nich chcemy przechowywać jego dane osobowe (z numerem PESEL), kontaktowe (telefon i opcjonalny adres Email), stanowisko oraz staż (wyliczany na podstawie daty zatrudnienia).
- 3) Pracownicy są zatrudniani na Umowę Zlecenie, B2B lub Umowę o pracę. Chcemy pamiętać datę zawarcia umowy, okres jej trwania (w miesiącach) i datę zakończenia. Dodatkowo na umowie pracownicy mają określone wynagrodzenie netto oraz brutto (obliczane na podstawie Netto).
- 4) Klientami wypożyczalni są osoby prywatne (włącznie z pracownikami). Dla klientów chcemy przechowywać dane osobowe (z numerem PESEL), kontaktowe (telefon i opcjonalny adres Email), ilość wypożyczonych pojazdów oraz datę rejestracji.
- 5) Dla pojazdu chcemy znać jego unikalny numer rejestracyjny, specyfikację (markę, model, rok produkcji, przebieg), datę przeglądu oraz cenę za dzień wypożyczenia.
- 6) Chcemy znać informację o stanie pojazdu. Konkretny pojazd może być dostępny, wynajęty, w serwisie lub wycofany.
- 7) Wypożyczalnia ma w swojej ofercie samochody osobowe oraz dostawcze. Podział ten jest kompletny.
- 8) Samochody osobowe mogą posiadać nadwozie typu Sedan, Kombi, SUV lub Hatchback, dodatkowo chcemy wiedzieć jaką liczbę miejsc ma dany pojazd i to, czy posiada klimatyzację.
- 9) Auta dostawcze mają określoną ładowność i pojemność.
- 10) Wszystkie pojazdy mogą być pokryte ubezpieczeniami. Dla ubezpieczenia chcemy znać unikalny numer polisy, jego datę ważności, cenę oraz firmę, która je dostarcza.
- 11) Dla wypożyczenia chcemy znać datę wypożyczenia, planowaną datę zwrotu, opcjonalną rzeczywistą datę zwrotu (samochód może jeszcze nie być zwrócony do wypożyczalni) oraz jego aktualny status (aktywne, zakończone, opóźnienie).

- 12) Za wypożyczenia wystawiane są faktury. System przechowuje jej datę wystawienia, kwotę netto oraz brutto (wyliczaną jako netto + VAT). Należy przechowywać również status rachunku - wystawiony, opłacony lub anulowany.
- 13) Pojazdy mogą być wysyłane do serwisu w celu naprawy usterek. Dla każdego zgłoszenia należy pamiętać datę, powód (maksymalnie 100 znaków) oraz aktualny status zgłoszenia (utworzone, w realizacji, zakończone).
- 14) Chcemy pamiętać zdarzenia dotyczące pojazdów w naszej flocie, w tym celu przechowujemy daty naszych zdarzeń wraz z opisem. System ma z góry określone typy zdarzeń (rezerwacja, wypożyczenie, zwrot, przegląd, serwis, ubezpieczenie).
- 15) Klienci mogą dokonywać rezerwacji pojazdów. Dla każdej rezerwacji chcemy pamiętać datę rozpoczęcia, zakończenia, utworzenia oraz status rezerwacji (oczekuje, zatwierdzona, anulowana).
- 16) System powinien wspomagać użytkowników w realizowaniu następujących funkcjonalności:
- a) dodawanie i rejestrowanie ubezpieczeń pojazdom,
 - b) logowanie się do systemu,
 - c) przeglądanie dostępnych pojazdów,
 - d) rezerwowanie pojazdów przez klientów,
 - e) anulowanie rezerwacji,
 - f) zatwierdzanie rezerwacji przez pracowników,
 - g) kończenie wypożyczeń i przyjmowanie zwrotów pojazdów,
 - h) rejestracja nowych klientów,
 - i) zarządzanie ubezpieczeniami pojazdów (przeglądanie, dodawanie, usuwanie),
 - j) wystawianie faktur,
 - k) przeglądanie historii pojazdów,
 - l) przeglądanie informacji o naprawach pojazdów,
 - m) dodawanie i usuwanie pojazdów z floty,
 - n) zarządzanie danymi pracowników i klientów,
 - o) obliczanie kosztów wypożyczenia zależnie od pojazdu:
 - Osobowy - zależnie od typu nadwozia,
 - Dostawczy - zależnie od pojemności i ładowności.

2. Diagramy przypadków użycia (wymagania funkcjonalne)

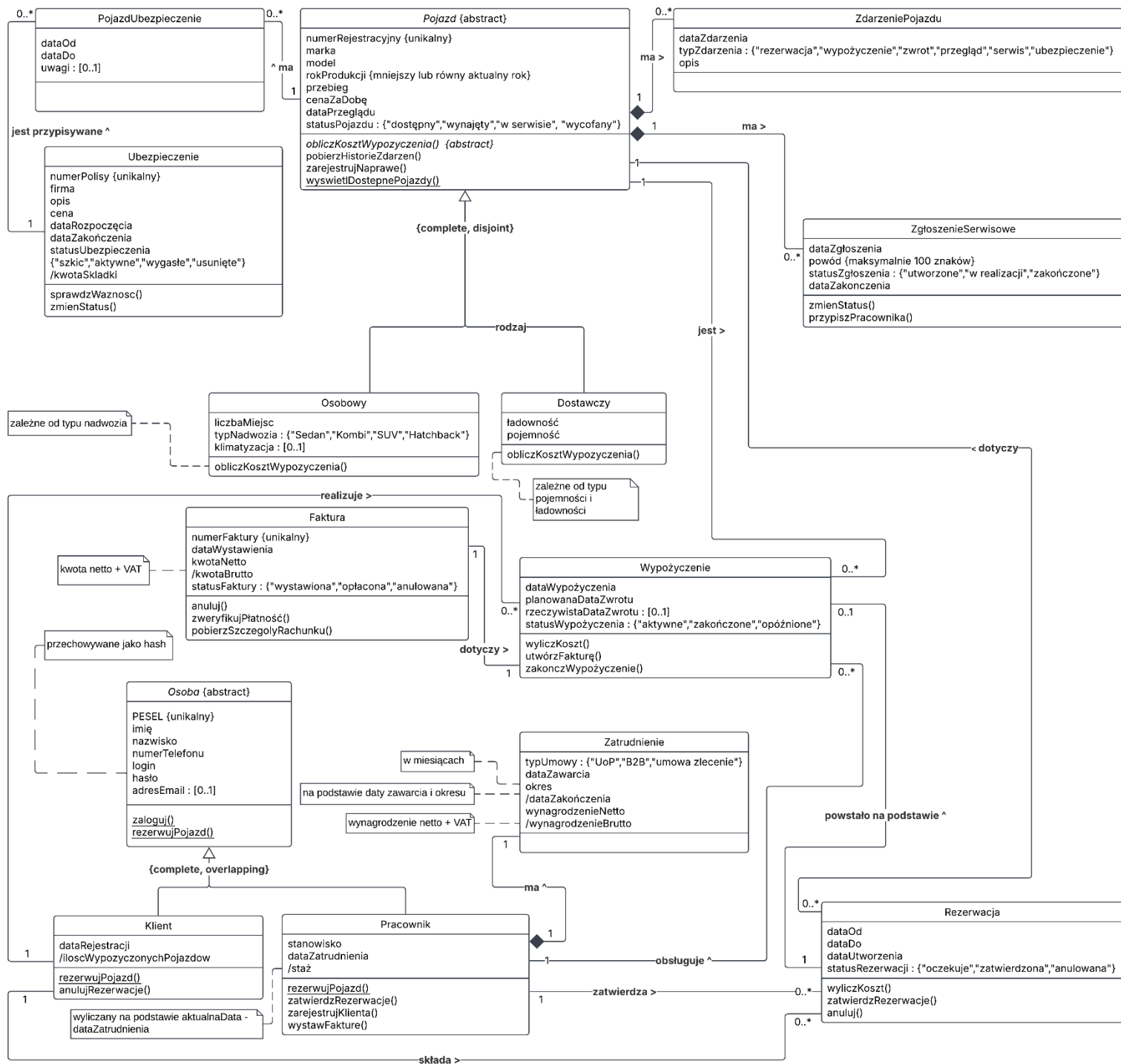


3. Wymagania niefunkcjonalne jako lista

- 1) Organizacyjne - system powinien wspierać codzienną pracę wypożyczalni pojazdów, w szczególności obsługę klientów, pracowników, pojazdów, rezerwacji, wypożyczeń, ubezpieczeń, serwisów oraz faktur.
- 2) Technologiczne - system powinien działać jako aplikacja desktopowa napisana w języku Java z wykorzystaniem JavaFX oraz relacyjnej bazy danych PostgreSQL.
- 3) Prawne - system powinien przetwarzać dane osobowe klientów i pracowników, takie jak PESEL, numer telefonu oraz adres e-mail, zgodnie z obowiązującymi przepisami dotyczącymi ochrony danych osobowych, w szczególności RODO.
- 4) Bezpieczeństwo - dostęp do systemu powinien wymagać zalogowania się na konto użytkownika. Hasła użytkowników powinny być przechowywane w postaci zahashowanej, a nie jako jawny tekst.
- 5) Kontrola dostępu - system powinien rozróżniać role użytkowników, takie jak klient oraz pracownik, aby ograniczać dostęp do funkcji zgodnie z zakresem obowiązków danego użytkownika.
- 6) Integralność danych - system powinien zapewniać unikalność kluczowych danych, takich jak numer PESEL, login użytkownika, numer rejestracyjny pojazdu, numer polisy czy numer faktury.
- 7) Wydajność - podstawowe operacje użytkownika, takie jak logowanie, przeglądanie pojazdów, wyświetlanie ubezpieczeń oraz zapisywanie danych, powinny wykonywać się w czasie nieprzekraczającym 3 sekund przy typowym obciążeniu systemu.
- 8) Niezawodność - system powinien informować użytkownika o błędach, takich jak brak połączenia z bazą danych, błędne dane logowania lub niepoprawnie wypełniony formularz.
- 9) Użyteczność - interfejs użytkownika powinien być czytelny, prosty i intuicyjny, tak aby pracownik wypożyczalni mógł szybko wykonywać podstawowe operacje bez konieczności długiego szkolenia.
- 10) Utrzymywalność - kod systemu powinien być podzielony na logiczne moduły, takie jak encje bazy danych, serwisy aplikacyjne, widoki interfejsu oraz klasy pomocnicze, aby umożliwić dalszą rozbudowę projektu.
- 11) Spójność danych - system powinien walidować dane wprowadzane przez użytkownika, np. poprawność zakresu dat, statusów, wymaganych pól oraz wartości liczbowych takich jak cena, przebieg czy wynagrodzenie.

4. Diagram klas – analityczny

cld "Karen't - wypożyczalnia pojazdów"



cld "Karen't - wypożyczalnia pojazdów"



6. Scenariusz przypadku użycia (jako wypunktowany tekst)

Nazwa przypadku użycia: Dodaj ubezpieczenie pojazdu

Opis: Pracownik chce dodać (przypisać) wybrane ubezpieczenie do konkretnego pojazdu w systemie wypożyczalni

Aktor główny: Pracownik

Aktor pomocniczy: System

Warunki początkowe:

- 1) Pracownik jest zalogowany do systemu.
- 2) W systemie istnieje wybrany pojazd.
- 3) W systemie istnieje przynajmniej jedno aktywne ubezpieczenie możliwe do przypisania.

Wynik końcowy: Ubezpieczenie zostaje przypisane do wybranego pojazdu (powstaje powiązanie Pojazd - Ubezpieczenie), a lista ubezpieczeń pojazdu zostaje zaktualizowana.

Scenariusz główny:

- 1) Pracownik wybiera w systemie opcję „*Zarządzaj ubezpieczeniami pojazdów*”
- 2) System wyświetla listę pojazdów
- 3) Pracownik wybiera pojazd z listy
- 4) System wyświetla listę ubezpieczeń przypisanych do wybranego pojazdu
- 5) Pracownik wybiera opcję „*Dodaj ubezpieczenie pojazdu*”
- 6) System wyświetla listę dostępnych ubezpieczeń (tylko aktywne)
- 7) System wyświetla formularz dodawania ubezpieczenia
- 8) Pracownik wybiera ubezpieczenie z listy oraz uzupełnia okres przypisania ubezpieczenia do pojazdu, dodaje ewentualne uwagi
- 9) Pracownik wybiera opcję zatwierdzenia danych
- 10) System sprawdza poprawność danych
- 11) System tworzy powiązanie ubezpieczenia z pojazdem (rekord *PojazdUbezpieczenie*)
- 12) System aktualizuje listę ubezpieczeń przypisanych do pojazdu
- 13) System wyświetla potwierdzenie dodania i kończy przypadek użycia

Scenariusze alternatywne:

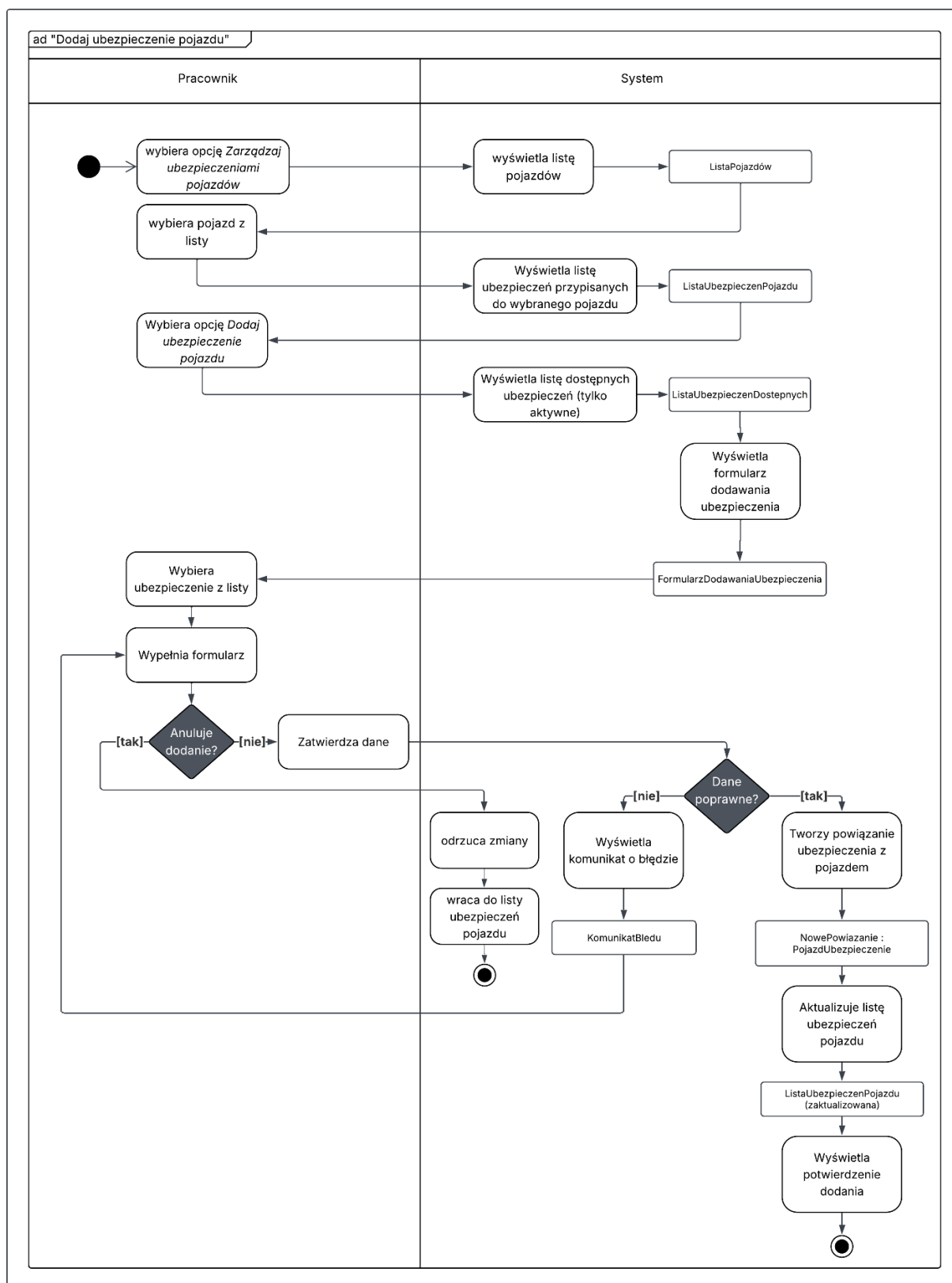
➔ Anulowanie dodawania ubezpieczenia

- W kroku 8 pracownik wybiera anulowania
 - 1) System odrzuca niezapisane dane
 - 2) System wraca do listy ubezpieczeń pojazdu
 - 3) Przypadek użycia kończy się bez zmian

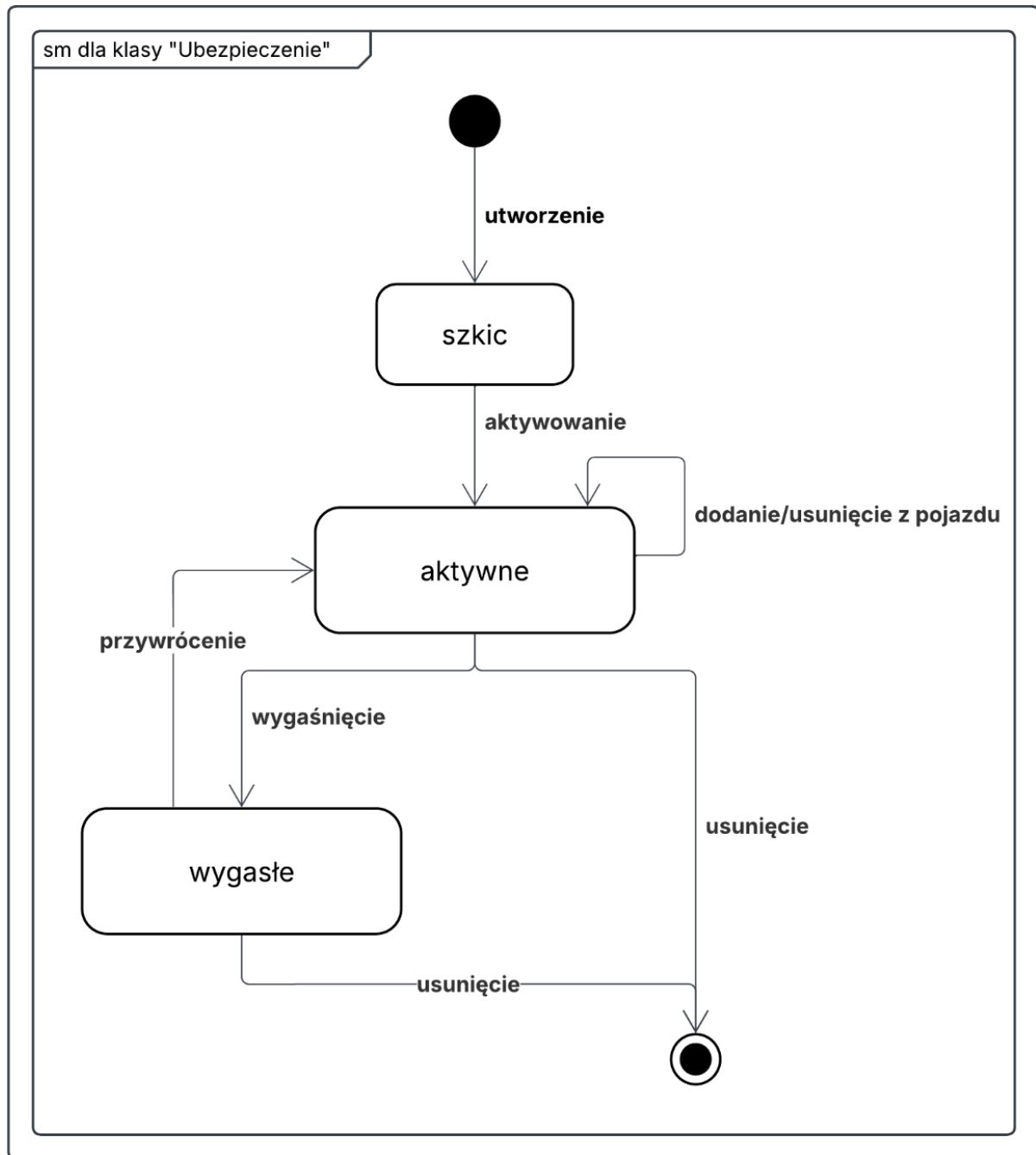
➔ Dane niepoprawne (błąd walidacji)

- W kroku 10 system wykrywa niepoprawne dane (np. brak wymaganych pól, błędny zakres dat, konflikt z zasadami systemu)
 - 1) System wyświetla komunikat o błędzie
 - 2) Pracownik poprawia dane w formularzu
 - 3) Następuje powrót do kroku 9 (ponowne zatwierdzenie danych)

7. Diagram aktywności dla przypadku użycia



8. Diagram stanu dla klasy



9. Projekt GUI

9.1 Scenariusz główny

Karen't – Panel pracownika



Zarządzanie

Zarządzaj ubezpieczeniami pojazdów >

Zatwierdź rezerwacje >

Historia pojazdów >

Zgłoszenia serwisowe >

 Powrót



Pojazdy

Status ▾

Szukaj numeru rej. 

NR REJ	MARKA	MODEL	STATUS	
KR1234A	Toyota	Corolla	Dostępny	>
WA9988X	Ford	Transit	W serwisie	>

← Powrót



Toyota Corolla (KR1234A)

Rok: 2022

Przebieg: 45 000 km

Status: Dostępny

Ubezpieczenia

Historia

Zdarzenia

Nowe ubezpieczenie

← Powrót



Ubezpieczenia pojazdu

POLISA	FIRMA	STATUS	OKRES	
12345	PZU	Aktywne	01.01 - 31.12	>
88721	Warta	Wygaste	01.01 - 31.12	>

Dodaj ubezpieczenie



Dodaj ubezpieczenie

Data od



RRRR-MM-DD

Data do



RRRR-MM-DD

Uwagi (opcjonalnie)

Anuluj

Zatwierdź



Ubezpieczenie pojazdu



Ubezpieczenie dodane

Ubezpieczenie zostało pomyślnie dodane do pojazdu.

Zamknij

POLISA			
12343			>
55433			>
88721			>
55433	PZU	26.04–26.04	01.01–31.12
55433	PZU	PZU	01.01–31.12

9.2 Scenariusze alternatywne

← Powrót



Dodaj ubezpieczenie

Data od

RRRR-MM

Uwagi (o

Czy chcesz zrezygnować z wypełniania wniosku o polisę?

Tak Nie

Anuluj Zatwierdź

← Powrót



Dodaj ubezpieczenie

Data od

RRRR-MM

Uwagi (o

 **Błąd**

Proszę uzupełnić poprawnie wszystkie pola formularza.

Wróć

Anuluj Zatwierdź

10. Omówienie decyzji projektowych i skutków analizy dynamicznej

- a) Aplikacja została zaprojektowana jako aplikacja desktopowa w języku Java, ponieważ jest to język najlepiej znany autorowi projektu, a interfejs użytkownika został zrealizowany przy użyciu JavaFX.
- b) Trwałość danych została zaprojektowana z wykorzystaniem relacyjnej bazy danych PostgreSQL. Do mapowania klas na tabele baz danych zastosowano Hibernate, dzięki czemu operacje na danych mogą być wykonywane na poziomie obiektów, bez konieczności ręcznego pytania zapytań SQL
- c) Każda klasa będzie wyposażona w metody dostępne (getter i setter) umożliwiające odczyt i modyfikację atrybutów. Pominęto ich odwzorowanie na diagramie w celu zachowania czytelności
- d) Niektóre atrybuty objęte są ograniczeniami np. {unikalny}, co oznacza, że ich wartości muszą być niepowtarzalne w ramach danej kolekcji obiektów
- e) Relacja pomiędzy klasą bazową Osoba a klasami pochodnymi Pracownik i Klient zostało zrealizowane w typie overlapping, dzięki czemu pojedynczy obiekt może pełnić jednocześnie rolę pracownika i klienta
- f) Wykorzystanie klas abstrakcyjnych wymusza implementację zdefiniowanych pól i metod w klasach pochodnych
- g) Pomiedzy klasami Pracownik oraz Zatrudnienie zastosowano kompozycję, oznacza to, że obiekt Zatrudnienie istnieje wyłącznie w kontekście konkretnego pracownika i przechowuje informacje dotyczące jego umowy
- h) W wielu miejscach zastosowano typy wyliczeniowe (enum), aby ograniczyć dopuszczalne wartości atrybutów do z góry określonego zbioru
- i) Na diagramie stanu dla klasy Ubezpieczenie zaznaczono, że dodanie lub usunięcie ubezpieczenia z pojazdu nie zmienia jego stanu. Ubezpieczenie pozostaje aktywne, ponieważ jego stan zależy przede wszystkim od dat ważności, a nie od powiązań z konkretnymi pojazdami
- j) Dziedziczenie overlapping na diagramie projektowym między klasą nadrzędną Osoba a klasami podrzędnymi Pracownik i Klient przedstawiono jako kompozycję, określając licznosc pozwalającą na to, aby Pracownik był również Klientem